Interaction-Aware Cooperative Residual Reinforcement Learning for Multi-USV Target Defense

Manuscript in preparation

***Abstract*—Multi-USV target defense requires individual maneuvers to form an effective team response against a more agile intruder. We develop an interaction-aware cooperative residual reinforcement learning framework that combines learned and Boids control through adaptive blending. Its policy mechanism conditions each vessel's blending coefficient on the motion relationships and concurrent candidate controls of its teammates. Its learning mechanism augments a joint critic with paired closed-loop return differences obtained by changing one vessel's current coefficient while retaining the other current decisions. Subsequent controls follow the same frozen policy on each branch's evolved state. The resulting value guides actor–critic policy optimization, while deployment uses the learned policy and a common safety filter. Matched learning, information, and model-budget controls are specified to evaluate the two mechanisms. Available fixed-policy diagnostics have not yet established a reliable task benefit from conditional-difference supervision; independent predictive-control results are retained as separate references.**

***Index Terms*—Multi-robot systems, unmanned surface vehicles, cooperative residual reinforcement learning, candidate-control interaction, conditional value learning.**

I. Introduction

Multi-USV target defense requires a team of defending vessels to intercept an intruder before it enters a protected region. Marine autonomy, territory guarding, and distributed pursuit connect this task to coordinated decision making and physical control [1], [2], [3]. When the intruder has a maneuverability advantage, defenders must use their spatial distribution and complementary approaches to sustain an effective interception [4], [5], [6], [7]. Their control decisions become increasingly coupled as vessels converge, making cooperative adaptation central to both regional defense and capture efficiency.

Reinforcement learning provides a means to acquire such maneuvers from task feedback. Decentralized pursuit, aerial pursuit–evasion, multi-USV hunting, and integrated marine decision making illustrate this approach [8], [9], [10], [11]. Mean embeddings, joint value functions, and message-conditioned representations support learning in variable-size teams [12], [13], [14], [15]. The present problem is how information about teammates' intended control should influence the use of each vessel's available control sources.

Rule-based coordination provides a structured control prior, while learned control can adapt maneuvers to the encounter. Residual learning and controller fusion combine these capabilities [26], [16], [17]. ARBoids [18] uses an adapter to blend learned and Boids actions as a function of individual observations, teammate relative states, and the vessel's own candidate actions. We retain these two control sources and study blending decisions conditioned additionally on the relationships between concurrent teammate candidates. The contribution concerns the decision value of those candidate relationships; teammate state information is already present in ARBoids.

The corresponding learning problem is to evaluate how a local blending change affects the subsequent team response. Counterfactual evaluation, closed-loop rollout, and model-based value expansion provide relevant foundations [23], [19], [24]. We compare two branches that differ in one vessel's current blending coefficient, keep all other current candidates and coefficients fixed, and resume the same feedback policy on each branch's own state. This supplies conditional return differences for the joint critic that trains the cooperative adapter. A common safety filter [20] separates physical execution from the attribution of task improvements; prediction in cooperative hunting [21] provides further context for using future consequences in decision making.

The method has three connected components. First, a cooperative residual framework preserves the complementary learned and Boids control sources. Second, candidate-interaction-conditioned adaptation and local-intervention value learning connect the blending representation to its task consequences. Third, matched learning and model-budget comparisons, together with agility, team-size, and opponent evaluations, test the mechanism's effect on defense. The current manuscript separates the implemented method and completed diagnostics from the learning results that remain to be established.

II. Task and Control Model

A. Dynamics, observations, and outcomes

Let N homogeneous defenders protect a circular target region of radius 15 m against one intruder. The numerical environment follows the released three-degree-of-freedom WAM-V model [18],

$$\dot{\eta}_i = R(\psi_i)\nu_i, \qquad M\dot{\nu}_i + C(\nu_i)\nu_i + D(\nu_i)\nu_i = \tau_i, \quad (1)$$

with the source current perturbations and a 0.05-s integration step. A command is held for four integration steps, giving a 0.2-s control period. Differential thrust lies in $[-500, 1000]$ N

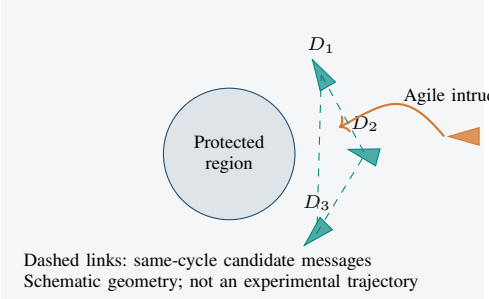


Fig. 1. Cooperative control composition for target defense. Dashed links carry concurrent candidate-control information; the drawing assigns no fixed maneuver roles. Defense succeeds through capture or survival to the 60-s deadline, with region entry and defender collision recorded separately.

per thruster; intruder agility scales its thrust bounds. Observations contain target-relative and intruder-relative geometry, measured intruder velocity, Boids features, and teammate distance–bearing pairs. Candidate messages add measured defender position, heading, translational velocity, yaw rate, and both current action candidates. No future disturbance, true intruder agility, or centralized training state is an actor input.

We use `paper-parameters-v1`: a run terminates at actual target entry ($d_{AT} \le 15$ m), a defender separation at or below 5 m, capture ($\min_i d_{iA} < 5$ m), or 60 s. This is also the priority order for simultaneous terminal events. Independent physical collision and breach flags are retained alongside that exclusive outcome. The deadline is an absorbing task termination, with zero bootstrap value.

B. Control sources and nominal composition

For the learning realization, Boids supplies pursuit, separation, alignment, and cohesion forces using the released parameters. A shared learned candidate $\ell_i \in (-1,1)^2$ maps to physical thrust $u_i^L = 750\ell_i + 250$. The mixed nominal control is

$$u_i^{\mathrm{nom}} = \theta_i u_i^L + (1-\theta_i)u_i^B, \qquad 0 \le \theta_i \le 1. \tag{2}$$

We define *control composition* as the state-dependent combination of the learned and Boids sources in the nominal command of (2). The candidate controls and team composition configuration are

$$\mathcal{A} = \big((u_i^L, u_i^B)\big)_{i=1}^N, \qquad \boldsymbol{\Theta} = (\theta_1, \dots, \theta_N). \tag{3}$$

Cooperative control composition seeks to match these local choices to inter-vessel interactions and the team task. Each blending coefficient describes the relative contribution of two sources to one nominal command: $\theta_i + (1-\theta_i) = 1$. There is no shared cross-vessel coefficient budget or constraint $\sum_i \theta_i = 1$. Local control authority is an interpretation of this relative contribution, not a transfer of actuator permissions.

For fixed candidate controls,

$$\frac{\partial u_i^{\mathrm{nom}}}{\partial \theta_i} = u_i^L - u_i^B. \tag{4}$$

Thus coefficient variation translates into command variation through candidate disagreement. The current implementation adds a fixed second-order CBF filter, with a 7-m barrier separation, the released relaxed quadratic-program settings, and identical physical saturation. This filter is absent from original ARBoids and is not one of the two proposed learning mechanisms. The complete method and its matched mechanism ablations share it; the original-method reference remains unfiltered. Writing $\mathbf{u}$ for the stacked thrust commands, execution in the filtered implementation is

$$\mathbf{u}^{\mathrm{exec}} = \mathcal{F}_{\mathrm{CBF}}(s, \mathbf{u}^{\mathrm{nom}}). \tag{5}$$

The blending coefficient therefore characterizes nominal composition, not the final motion's proportional contribution from each source. Pursuit, lateral interception, and region maintenance are functions to identify in recorded trajectories. They are not roles assigned by large or small coefficients: Boids itself includes pursuit, and the learned branch receives task, formation, and collision feedback. Relaxed CBF feasibility is evaluated through measured collision outcomes.

C. Task reward

The defender reward retains the paper's task, formation, and collision terms. Target entry yields -100 to each defender; capture yields 100 to a defender within the capture radius, 50 within three capture radii, and zero task reward otherwise. Each colliding defender receives one -50 penalty. Writing $e_i = (p_A - p_i)/\|p_A - p_i\|$, $q = \sum_i e_i$, and $e_T = p_A/\|p_A\|$, the formation term is

$$r_i^{\mathrm{form}} = \tfrac{1}{2} e_T^\top q / \max(\|q\|, \epsilon) - \|q\|/N. \tag{6}$$

Terms are evaluated on terminal transitions too, with the exact released boundary comparisons retained in code. The joint learner uses $\bar{r} = N^{-1}\sum_i r_i$. Return, defense success, and capture time are distinct quantities throughout the evaluation.

III. Learning Cooperative Control Composition

The primary research object remains the original learned–Boids fusion. The two mechanisms below are implemented but have not yet established stable incremental task gains. A failed training run alone does not diagnose an action-space limitation.

A. Candidate-control interaction-conditioned blending

The first stage independently generates learned candidates from the shared ARBoids observation encoder. Its task and Boids encoders and masked mean teammate embedding feed the retained 512-unit decision backbone. Reparameterized Gaussian samples are squashed by $\tanh$ to form ℓ_i. The second stage exchanges candidates from the same control instant. For each directed pair, a shared two-layer, 128-unit encoder receives

$$z_{ij} = (\delta p_{ij}^i/60, \sin\delta\psi_{ij}, \cos\delta\psi_{ij}, \delta v_{ij}^i/5, \delta r_{ij}, \ell_i, b_i, \ell_j, b_j), \tag{7}$$

where $b_i = (u_i^B - 250)/750$ and superscript i denotes rotation into defender i's heading frame. For valid non-self neighbors

$\mathcal{N}_i$, the candidate-interaction message and conditional blending policy are

$$h_i^{\mathrm{rel}} = \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} f_\kappa(z_{ij}), \qquad \theta_i \sim \pi_\phi^g(\cdot \mid h_i, \ell_i, b_i, h_i^{\mathrm{rel}}). \tag{8}$$

An empty neighbor mask yields a zero message. This representation conditions composition on current candidate relations; it imposes no requirement that different defenders use different coefficients. The gate mean adds a message-dependent residual to the original adapter logit; the residual initializes at zero. A learned gate log standard deviation initializes at $\log(0.1)$.

With h_i the backbone feature and $\epsilon_i^g \sim \mathcal{N}(0,1)$,

$$z_i^g = \mu_i^g(h_i, h_i^{\mathrm{rel}}, \ell_i, b_i) + \sigma_i^g(h_i, h_i^{\mathrm{rel}})\epsilon_i^g, \quad \theta_i = \frac{\tanh z_i^g + 1}{2}. \tag{9}$$

Thus the joint density factorizes into a proposal stage and a conditional gate stage,

$$\Pi_\phi(\ell, \mathbf{\Theta} \mid o) = \prod_i \pi_\phi^L(\ell_i \mid o_i) \prod_i \pi_\phi^g(\theta_i \mid h_i, h_i^{\mathrm{rel}}, \ell_i, b_i). \tag{10}$$

The gate log density includes both transformations:

$$\log \pi_i^g = \log \mathcal{N}(z_i^g; \mu_i^g, \sigma_i^g) - \log(1 - \tanh^2 z_i^g) + \log 2. \tag{11}$$

We evaluate the log-Jacobian in a stable softplus form. Gate losses backpropagate through the candidate messages when the proposal branch is unfrozen. Deterministic evaluation uses the transformed Gaussian means in both stages.

B. Joint configuration value and real-transition learning

For policy Π, let $\mathcal{Q}^\Pi(s, \mathcal{A}, \mathbf{\Theta})$ denote the team's soft value conditioned on the current state, candidate controls, and complete composition configuration. Holding $\mathcal{A}$ fixed distinguishes a change in composition from an arbitrary replacement of the candidate controls. A local value can also be defined by averaging over teammates' coefficients under their policy. Retaining the full configuration enables the critic to represent differences across those teammate choices instead of only their average. This is value-level configuration interaction, distinct from the candidate-interaction message in (8).

In the implementation, u_i^L is determined by ℓ_i and the Boids command by s, so the same value is represented as $Q^\Pi(s, a)$ with $a = (\ell, \mathbf{\Theta})$. The transition includes the common CBF filter and subsequent vessel dynamics.

Each of two critics encodes a node containing a defender's centralized kinematic state, normalized composed nominal thrust, left–right thrust difference, and Boids candidate. The action coordinates are $v_i = \theta_i \ell_i + (1 - \theta_i) b_i$, where the corresponding physical thrust is $750 v_i + 250$ N and b_i is the normalized Boids candidate. Equivalent proposal–gate factorizations therefore receive exactly the same value, as they produce the same input to the CBF; the current action's entropy is outside Q. The audited state representation additionally includes the measured ground velocity used by the CBF and the vessel's normalized position in the simulator's obstacle traversal order. Relative-to-water velocity alone does not determine the CBF input, and the released APF opponent is order dependent. These four additional node scalars correct the conditioned state without changing either controller. A symmetric pair encoder takes the sum and absolute difference of node embeddings and pair distance. Masked means pool nodes and non-self pairs. The critic head receives both pools, the intruder state, agility, remaining task time, and $N/7$, and outputs one team value. Shared encoders accept two through seven defenders, but invariance to changes in the simulator's traversal order is not imposed. Centralized information is restricted to critic training. This corrected representation is versioned as `execution-v2`; historical checkpoints retain their original `dynamics-v1` interpretation and cannot silently resume under the new representation.

One replay record represents one complete team control cycle: observations, public kinematics, centralized state, proposals, all blending coefficients, actual filtered thrust, individual and mean rewards, next observations and states, and the terminal flag. Critic actions are $(\ell, \mathbf{\Theta})$; filtered thrust is recorded for audit.

The entropy term uses the same per-defender scale as the mean team reward. For stage z, define

$$e_\phi(o, a; z) = \frac{\mathbb{I}[z = \mathrm{joint}]}{N} \sum_{i=1}^{N} \log \pi_\phi^L(\ell_i \mid o_i). \tag{12}$$

During gate learning, the frozen proposal distribution supplies exogenous candidates and the gate optimizes physical task return. Joint learning regularizes the trainable proposal distribution at the per-defender scale. Gate sampling supports exploration, while its mean and variance learn through task value without a gate-density reward. For $a = (\ell, \mathbf{\Theta})$, our real-data evaluator and soft actor–critic [22] policy updates use

$$y = \bar{r} + \gamma(1 - d) \left[\tfrac{1}{2} \sum_{k=1,2} \bar{B}_k(s', a') - \alpha e_\phi(o', a'; z) \right], \tag{13}$$

$$L_B = \sum_k \mathbb{E}[(B_k(s, a) - \mathrm{sg}(y))^2], \tag{14}$$

$$L_{\mathrm{TD}} = \sum_k \mathbb{E}[(Q_k(s, a) - \mathrm{sg}(y))^2], \tag{15}$$

$$L_\pi = \mathbb{E}[\alpha e_\phi(o, a; z) - \min_k Q_k(s, a)]. \tag{16}$$

The real-data evaluator averages its two heads for bootstrapping; policy improvement retains the main-critic minimum. Full, Same-info, No-candidate-interaction, Short, and Model-return share this choice. Truncated rollout tails use the same evaluator average, while the registered conditional-value diagnostics retain their original definition. We use $\gamma = 0.99$, target smoothing 0.005, and minibatches of 4096 joint transitions. Adam learning rates are 10^{-4} for critics and 10^{-5} for the actor and temperature. Proposal temperature starts at 0.2 and adapts during joint learning to a per-defender entropy target of -2; the gate stage has zero entropy cost and no temperature update. Each matched joint-learning arm has a main twin critic Q and an independent twin bootstrap evaluator B, both with the same 512/128 widths and initialization. After initialization from complete real-episode returns, the evaluator receives only L_B on real transitions, and its lagged copy follows $\bar{B} \leftarrow$

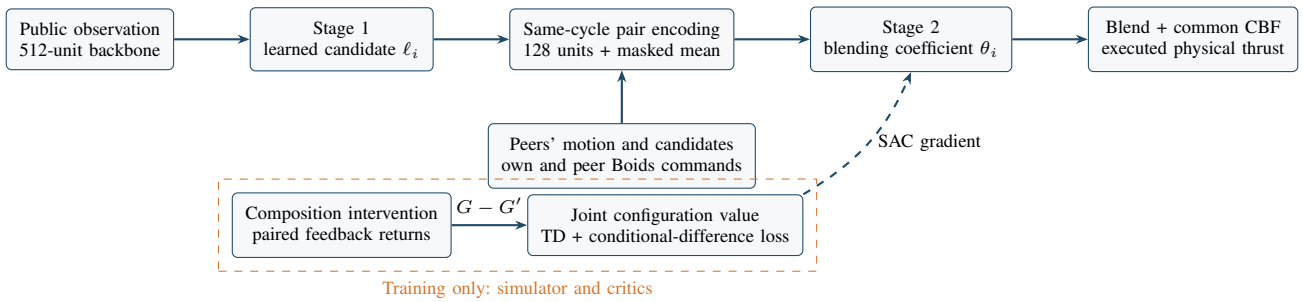


Fig. 2. The coupled policy and learning mechanism. Candidate-control relations condition local blending; paired composition interventions supervise conditional differences in the joint configuration value. Actor–critic updates connect that evaluation to the policy. Deployment uses the actor and common CBF.

$(1-\tau)\bar{B}+\tau B$. The main critic receives the real-transition loss and the arm's prescribed auxiliary loss. Auxiliary gradients and main-critic parameters never update B or $\bar{B}$. Same-info retains the same evaluator updates; without auxiliary supervision its main and bootstrap critics remain identical. The additional evaluator updates are counted separately.

C. Local composition intervention and conditional return difference

Let $\boldsymbol{\Theta}^{(i)} = (\vartheta_i, \boldsymbol{\Theta}_{-i})$ mean replacing component i by a reference coefficient ϑ_i while retaining the original indexing. The conditional value difference of this local composition change is

$$\Delta_i^{\Pi}(s, \mathcal{A}, \boldsymbol{\Theta}; \vartheta_i) = \mathcal{Q}^{\Pi}(s, \mathcal{A}, \boldsymbol{\Theta}) - \mathcal{Q}^{\Pi}(s, \mathcal{A}, \boldsymbol{\Theta}^{(i)}). \tag{17}$$

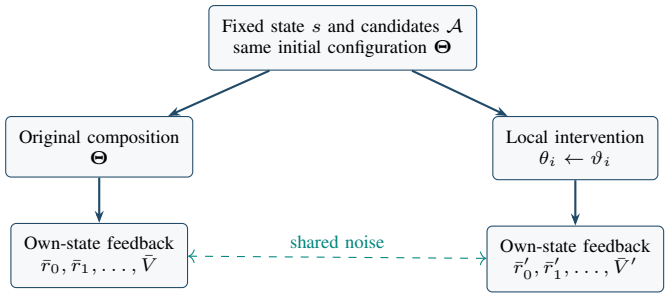
For a captured training state, sample one common first joint action and a defender index i. Draw ϑ_i uniformly from $\{0, 0.5, 1\}$. The alternate first action $a^{(i)}$ fixes s, $\mathcal{A}$, and $\boldsymbol{\Theta}_{-i}$ and changes only θ_i. Both branches start from separate complete copies of the same simulator state and share the same external random-number stream and policy sampling noise. From the second control cycle onward, every defender recomputes both policy stages using its own branch's evolved observations under the same frozen policy. The intervention is applied once; it does not fix the subsequent coefficients. The CBF, task reward, discount, and termination rules are identical in both branches.

For label batch b, freeze copies $(\Pi_b, \alpha_b, \bar{B}_b)$ of the policy, temperature, and tail evaluator. Let H be the rollout horizon and $K \le H$ the executed number of steps. Define

$$G_H(s,a) = \sum_{k=0}^{K-1} \gamma^k \bar{r}_k - \alpha_b \sum_{k=1}^{K-1} \gamma^k e_{\phi_b}(o_k, a_k; z) + \mathbb{I}[\text{not terminal at } K]\gamma^K \bar{V}(s_K), \tag{18}$$

$$\bar{V}(s_K) = \tfrac{1}{2} \sum_{j=1,2} \bar{B}_{b,j}(s_K, a_K) - \alpha_b e_{\phi_b}(o_K, a_K; z). \tag{19}$$

The prescribed first action has no entropy contribution, consistent with the soft Q convention. The subsequent action sample used for a truncated tail is shared by noise across branches. The actor, temperature, and real-data tail evaluator are frozen for an entire label batch, and the resulting targets



$$\Delta G_H = G_H(s,a) - G_H(s,a^{(i)})$$

Fig. 3. A local composition intervention fixes the current state, both candidate controls of every defender, and all other current coefficients. Later actions follow the same policy on each branch's own state. The label estimates a conditional soft-return difference for the composition variable.

stop gradients. Full, No-candidate-interaction, and Model-return use $H = 300$ and reach task termination, so their labels contain no learned tail; Short uses $H = 10$ and the truncated expression above. The sampled training target is

$$\widehat{\Delta G}_{H,i} = \frac{1}{R} \sum_{r=1}^{R} \left[G_H^{(r)}(s,a) - G_H^{(r)}(s,a^{(i)}) \right]. \tag{20}$$

For training, $R = 8$: the state, first action, and intervention remain fixed across repetitions; future environment and policy noise are independent across repetitions and shared within each pair. The label combines closed-loop simulated rewards with a frozen value tail when the horizon is truncated. Its within-state Monte Carlo standard error is recorded. Thus the supervision contains simulated consequences of the composition change, rather than only a difference between the current critic's own predictions. The ordinary-return control uses the mean of each branch over the same eight repetitions.

D. Composition-specific conditional value supervision

The complete method adds

$$L_\Delta = \sum_{j=1}^{2} \mathbb{E}\left[\left(Q_j(s,a) - Q_j(s,a^{(i)}) - \mathrm{sg}\{\widehat{\Delta G}_{H,i}\} \right)^2 \right] \tag{21}$$

to the real-transition loss: $L_Q = L_{\mathrm{TD}} + 0.1L_\Delta$. The ordinary model-return control uses

$$L_G = \tfrac{1}{2}\sum_j \mathbb{E}\big[(Q_j(s,a) - \operatorname{sg} G_H(s,a))^2 + (Q_j(s,a^{(i)}) - \operatorname{sg} G_H(s,a^{(i)}))^2\big]. \tag{22}$$

Both objectives consume both branches. Each method generates labels under its own current frozen policy. The random-seed rule, snapshot sampling, intervention distribution, pair count, and horizon are matched; trajectories need not coincide after policies diverge. This avoids assigning another policy's continuation return to a method's own conditional value. Real transitions learn task value, paired consequences distinguish the conditional effect of local composition, and the actor objective transmits that evaluation to the blending policy. This supervision targets the composition variable and its closed-loop consequences within actor–critic learning; it does not introduce a separate role or responsibility allocator.

The long variants run to actual task termination, for at most $H = 300$ control cycles (the remaining portion of the 60-s episode); the short variant uses $H = 10$ (2 s). After critic warmup, every 1000 real transitions produces 64 fixed root interventions drawn from a rolling pool of 256 snapshots sampled every 16 transitions. Each root uses eight paired futures. The 64 averaged labels are reused between refreshes alongside the 4096-transition real minibatch, a ratio of $1/64$; rollout generation does not cover all 4096 samples at every update. A new batch replaces the previous labels. The exact conditional-return interpretation refers to the generating snapshot Π_b; reusing labels while the actor changes introduces a policy-lag approximation. Fixed-policy critic diagnostics hold this continuation policy constant. No auxiliary rollout transition enters the ordinary replay. The implementation uses the complete training simulator for labels. Long-label error includes finite-sample variability; Short additionally depends on a bootstrapped tail. Real transitions, simulated transitions, gradient updates, and wall time are counted separately.

E. Training procedure and curriculum

At each control cycle, the shared policy generates learned candidates, combines them with Boids candidates and allowed teammate messages, and samples the blending coefficients. The common CBF maps nominal controls to executed thrust, and the complete team transition enters replay. Periodically sampled training states provide single-vessel interventions and paired closed-loop consequences. Real-transition TD and the declared auxiliary objective update the joint critic; the actor is optimized through the critic. The policy, return convention, and tail evaluator are held fixed within each label batch.

The initial stage freezes the proposal branch and trains the adapter. Subsequent joint learning updates both branches under the same control structure. The agility curriculum uses $L_{\mathrm{agi}} \sim U(\bar{L}_{\mathrm{agi}} - 0.5, \bar{L}_{\mathrm{agi}} + 0.5)$ with $\bar{L}_{\mathrm{agi}} = 2.0 + 0.25k$, $k = 0, 1, 2, 3$. All compared arms share the initial policy, real-transition budget, update schedule, CBF, and task protocol. The saved configuration specifies the stage lengths, rollout horizon, repetition count, and any evaluator warmup. Source hashes and raw episode outcomes bind results to that configuration.

TABLE I
EARLIER ONLINE-SELECTION EVIDENCE MOTIVATING COMPOSITION AND LONG-TERM TASK EVALUATION; ORIGINAL 80-S PROTOCOL

Original method	Success	Capture	Collision	T_{80} (s)
Unfiltered ARBoids	146	146	878	70.248
ARBoids+CBF	1000	998	0	21.095
Matched short selector	993	992	0	20.681
Strong fixed 2-s selector	1008	1007	0	19.423
Adaptive long selector	1024	1022	0	17.087
Guard-oriented selector	1020	954	0	27.573
Fixed 1-s long selector	1024	1022	0	16.907

IV. Experimental Protocol and Available Evidence

A. Experimental setup

The main learning experiments use the released USV dynamics, the common CBF, and the 60-s paper termination protocol. Capture, target breach, inter-defender collision, and timeout are reported separately; every non-capture episode receives the 60-s cap when comparing capture time. Training and evaluation configurations specify team size, intruder agility, initialization, and disturbance streams. Paired comparisons share scenarios and external disturbances, while independent confirmation uses previously unused streams. Computation records distinguish real environment transitions, model transitions, optimizer updates, label generation, and deployment latency.

B. Cooperative behavior and prior task evidence

Earlier online-selection experiments motivate both the research object and the learning signal. Under the same instantaneous risk objective, independent and joint coefficient selection achieved 403/512 and 433/512 successes, respectively, with collisions decreasing from 55 to 29. The comparison shows that organizing the same local control sources differently can change team outcomes, motivating cooperative control composition.

Table I retains the preceding six-defender, 1024-scenario results under the 80-s source protocol. ARBoids+CBF, the matched short selector, and the adaptive long selector all recorded zero collisions, while their capped capture times were 21.095, 20.681, and 17.087 s. The long selector reduced capped time by 17.38% relative to the matched short selector. These results motivate evaluating sustained task consequences in addition to immediate safety. The long selector chose online among five whole-team feedback candidates using existing policies. The new experiments ask whether the proposed composition-specific learning mechanism can acquire useful choices within the policy; its results are reported separately under the 60-s protocol.

The later 22-candidate fixed-guard and 43-candidate adaptive joint predictors remain independent strong references. In two frozen six-defender banks, the adaptive predictor captured 512/512 intruders without breaches or collisions. The fixed-guard predictor also captured every intruder and had lower

TABLE II
MATCHED CONTROLS FOR THE ORIGINAL RESIDUAL LEARNING MECHANISM.

Method	Question addressed
Original ARBoids, no CBF	Complete-method task comparison with the original adaptive residual policy
ARBoids + CBF, continued training	Supplementary control with the added execution filter and matched additional training budget
Same-info, ordinary TD	Increment from paired conditional supervision
No peer candidates	Increment from explicit teammate candidate relationships
TD + absolute returns	Difference supervision versus the same model-return budget
Short conditional returns	Contribution of longer closed-loop consequences
Full	Combined candidate-interaction and conditional-value learning

mean capped times in both banks: 19.814 versus 21.103 s, and 20.205 versus 20.523 s. These are predictive-controller outcomes. They do not establish a learned-policy gain, and neither predictor replaces Boids in the proposed residual policy.

C. Learning performance and mechanism controls

The primary task reference is original ARBoids without CBF. ARBoids with the added CBF and matched continued training is reported separately as a supplementary control; superiority over it on every metric is not a prerequisite for the proposed contributions. Table II states the comparison roles. The complete-method contrast with original ARBoids includes execution-filter and training-budget differences, so it alone cannot identify either learning mechanism's contribution. Matched ablations retain the same execution filter and learning budget. The candidate-interaction comparison keeps teammate motion information and the vessel's own candidates available in both arms. Ordinary-TD candidate-on/off probes isolate the policy mechanism before interpreting its combination with auxiliary supervision. Task learning curves are compared at fixed environment-step checkpoints, with model and optimization costs reported separately. Candidate information, conditional-value accuracy, and task improvements are distinct claims. A lower auxiliary loss or a changed gate distribution alone does not establish a task gain. Checkpoint selection uses development data; independent learning repetitions and final held-out scenarios establish repeatability after a promising configuration is fixed.

D. Completed fusion and value-learning diagnostics

The following critic experiments used the pre-audit state representation. A subsequent consistency audit identified missing CBF velocity information and an invalid permutation assumption for the order-dependent APF opponent. Their corrections pass analytic, finite-difference, and execution regression checks; task-level gains from the corrected representation have not yet been evaluated. Thus the historical results diagnose those checkpoints and do not establish that the two original mechanisms are intrinsically ineffective. The original learned–Boids opportunity test used 48 independent source-policy states: 16 each with three and six defenders at agility 2.25, and eight each with six defenders at agility 4 and 6. All 48 admit a legal single-vessel blending change that alters actual post-CBF thrust. Each candidate changes only the current focal coefficient; subsequent controls resume the same frozen source policy on each branch's own state. Four shared search futures selected a candidate, followed by eight new paired futures. Task-selected capped-time differences were -0.497, -1.427, -0.919, and -2.084 s in the four conditions, respectively; every condition's exploratory state-bootstrap interval includes zero. The descriptive equal-state average is -1.142 s, with a post-hoc stratified 95% interval of $[-2.418, 0.072]$ s. The within-state standard deviation of a single paired time difference is 12.521 s. Reward-selected interventions give a similar uncertain mean change, -1.228 s. These outcomes establish executable action variation, but neither stable improvement nor impossibility of improvement.

A separate frozen-critic direction probe evaluates the v8 Full 10k checkpoint without an actor update. At each state, it chooses the vessel with largest absolute derivative of $\min(Q_1, Q_2)$ with respect to its coefficient and tests a clipped 0.1 change in that direction. An initial source-state probe includes six-vessel conditions, outside this critic's three-vessel training distribution; those cases are only extrapolation diagnostics. The subsequent check uses one uniformly sampled pre-action state from each of 64 new complete episodes of the checkpoint's own stochastic policy, under its three-vessel, agility-2.25 ± 0.5 training conditions. Sixteen paired complete futures per action use the same frozen continuation policy and training return convention. Predicted mean improvement is 0.163, versus measured improvement 0.039 (95% interval $[-0.327, 0.424]$). The corresponding capped-time difference is -0.004 s ($[-0.284, 0.274]$). Conditional-value MAE is 0.670, versus 0.608 for a zero-difference predictor.

In 35 of these 64 states, the proposed increase changes joint executed thrust by less than 1 N; no nominal change greater than 1 N is reduced below that threshold by the CBF. Coefficients near zero or one account for 32.8% and 91.7% of the 192 vessel-level outputs of the 5k and 10k checkpoints on these same states. Thus the observed failure includes severe gate drift and many almost ineffective outward changes. This coordinate probe is not a shared-parameter actor update and does not isolate the cause of that drift.

The subsequent matched critic experiment freezes the original deterministic policy. Three defenders face agility 2.25 ± 0.5; 256 training episodes provide 24195 real transitions, and 128 other episodes supply independent test roots. One uniformly sampled pre-action state per episode is evaluated with its current action and three single-vessel alternatives, $\theta_i \in \{0, 0.5, 1\}$, over eight complete paired futures. All arms share the same 512/128 twin-critic architecture, real minibatches, TD-only lagged targets, and model-return samples. A common 1500-update state-value warmup precedes 3000 fixed-policy critic updates. Auxiliary weight is 0.1. Initializations 42 and 101 share the dataset and are not independent data replications. No actor is updated or test-based checkpoint selected. All learned difference errors exceed the zero predictor (Table III), and all exploratory paired time intervals include zero. Difference supervision does not outperform matched absolute supervision. The estimated noise variance of an eight-future

TABLE III
FIXED-POLICY CRITIC DIAGNOSTIC ON 128 INDEPENDENT STATES. TIME CHANGES FOLLOW ONE SELECTED INTERVENTION AND THEN THE FROZEN SOURCE POLICY.

Seed	Supervision	ΔQ MAE	ΔT_{60} (s)
42	TD	2.505	+0.111
42	TD + absolute	2.469	+0.217
42	TD + difference	2.483	+0.134
101	TD	2.499	+0.134
101	TD + absolute	2.429	+0.206
101	TD + difference	2.459	+0.080
—	Zero difference	2.088	—

difference mean is 11.403, compared with an observed second moment of 12.847. These results identify difficult conditional-value generalization even without actor drift; they do not establish its unique cause. The experiment uses 681244 simulated steps. Critic-selected actions use centralized state and are diagnostic probes, not a deployed learned actor.

E. Attacker agility

The original evaluation varies intruder agility at fixed team size and reports defense success, capture, collision, and capped time. The intended question is whether candidate-conditioned blending maintains effective coordination as individual pursuit becomes harder. Existing predictor curves retain their method and protocol labels; learned-policy curves require corresponding completed learning runs.

F. Generalization to unseen team sizes

The shared policy and masked aggregation permit evaluation across two through seven defenders after three-defender training. Generalization is assessed by the same task metrics and matched baselines, rather than inferred from variable-size network inputs. Conditions outside the training team size are reported separately from in-distribution mechanism checks.

G. Learning-based attacker and high-fidelity execution

The planned opponent extension follows the original alternating attacker–defender training organization, using the same opponent budget for the compared defender methods. VRX evaluation uses the learned adapter, its declared messages, and the common physical-thrust mapping. Policy-generated trajectories and blending coefficients connect the learned variables to cooperative behavior. Existing VRX runs remain evidence for their original controllers; final claims about the proposed policy await its own completed runs.

V. DISCUSSION

The research question is whether candidate-control relationships and intervention-conditioned value information can improve cooperative residual policy learning. The current frozen-policy results show that legal blending changes affect executed thrust, but have not established stable task gains or an advantage of difference supervision over matched absolute returns. These findings determine which learning claims remain open; they do not demonstrate that the complete residual policy lacks useful control capacity.

The intervention protocol evaluates one current blending decision followed by feedback. Finite return differences and local actor gradients are related learning objects, but accurate finite differences alone do not guarantee useful parameter updates. Independent control and task measurements are therefore needed to connect critic learning to policy improvement. The predictive references demonstrate strong cooperative control under their own decision rules and provide comparison data without becoming a learned high-level policy or changing the present method's decision variables.

VI. CONCLUSION

This manuscript develops an interaction-aware cooperative residual learning method for multi-USV target defense. Candidate-control relationships condition the learned–Boids adapter, and single-vessel blending interventions provide paired closed-loop consequences for a joint critic. The framework retains a direct actor-based execution path and a common safety filter. Final empirical conclusions about the two learning mechanisms depend on the corresponding matched learning, independent replication, and task evaluations; completed predictive results remain separately attributed reference evidence.

REFERENCES

[1] Y. Qiao et al., "Survey of deep learning for autonomous surface vehicles in marine environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3678–3701, 2023, doi: 10.1109/TITS.2023.3235911.

[2] H. Fu and H. H.-T. Liu, "Guarding a territory against an intelligent intruder: Strategy design and experimental verification," *IEEE/ASME Trans. Mechatron.*, vol. 25, no. 4, pp. 1765–1772, 2020, doi: 10.1109/TMECH.2020.2996901.

[3] Y. Chen et al., "A distributed pipeline for collaborative pursuit in the target guarding problem," *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2064–2071, 2024, doi: 10.1109/LRA.2024.3349977.

[4] R. Yan et al., "Multiplayer homicidal chauffeur reach-avoid games: A pursuit enclosure function approach," *Automatica*, vol. 167, Art. no. 111770, 2024, doi: 10.1016/j.automatica.2024.111770.

[5] X. Fang, C. Wang, L. Xie, and J. Chen, "Cooperative pursuit with multi-pursuer and one faster free-moving evader," *IEEE Trans. Cybern.*, vol. 52, no. 3, pp. 1405–1414, 2022, doi: 10.1109/TCYB.2019.2958548.

[6] X. Guo, R. Cui, and W. Yan, "Pursuit-evasion games of marine surface vessels using neural network-based control," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 1, pp. 18–27, 2025, doi: 10.1109/TSMC.2023.3347044.

[7] M. Zhou, Z. Wang, J. Wang, and Z. Cao, "Multi-robot collaborative hunting in cluttered environments with obstacle-avoiding Voronoi cells," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 7, pp. 1643–1655, 2024, doi: 10.1109/JAS.2023.124041.

[8] C. de Souza et al., "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4552–4559, 2021, doi: 10.1109/LRA.2021.3068952.

[9] R. Zhang et al., "Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7900–7909, 2023, doi: 10.1109/TNNLS.2022.3146976.

[10] S. Xue et al., "Multi-agent self-attention reinforcement learning for multi-USV hunting target," *Neural Netw.*, vol. 189, Art. no. 107574, 2025, doi: 10.1016/j.neunet.2025.107574.

[11] X. Lin, P. Szenher, Y. Huang, and B. Englot, "Distributional reinforcement learning based integrated decision making and control for autonomous surface vehicles," *IEEE Robot. Autom. Lett.*, vol. 10, no. 2, pp. 1194–1201, 2025, doi: 10.1109/LRA.2024.3518839.

[12] M. Hüttenrauch, A. Šošić, and G. Neumann, "Deep reinforcement learning for swarm systems," *J. Mach. Learn. Res.*, vol. 20, no. 54, pp. 1–31, 2019. https://jmlr.org/papers/v20/18-476.html

[13] T. Rashid et al., "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, no. 178, pp. 1–51, 2020. https://jmlr.org/papers/v21/20-081.html

[14] Z. Ma, Y. Luo, and J. Pan, "Learning selective communication for multi-agent path finding," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1455–1462, 2022, doi: 10.1109/LRA.2021.3139145.

[15] Q. Li, W. Lin, Z. Liu, and A. Prorok, "Message-aware graph attention networks for large-scale multi-robot path planning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5533–5540, 2021, doi: 10.1109/LRA.2021.3077863.

[16] K. Rana et al., "Bayesian controller fusion: Leveraging control priors in deep reinforcement learning for robotics," *Int. J. Robot. Res.*, vol. 42, no. 3, pp. 123–146, 2023, doi: 10.1177/02783649231167210.

[17] S. H. Jeon, H. J. Lee, S. Hong, and S. Kim, "Residual MPC: Blending reinforcement learning with GPU-parallelized model predictive control," *IEEE Trans. Robot.*, vol. 42, pp. 3469–3488, 2026, doi: 10.1109/TRO.2026.3721669.

[18] J. Tao, T. Shen, D. Zhao, and F. Zhang, "ARBoids: Adaptive residual reinforcement learning with Boids model for cooperative multi-USV target defense," *IEEE Robot. Autom. Lett.*, vol. 11, no. 3, pp. 3637–3644, 2026, doi: 10.1109/LRA.2026.3662620.

[19] S. Bhattacharya et al., "Multiagent reinforcement learning: Rollout and policy iteration for POMDP with application to multirobot problems," *IEEE Trans. Robot.*, vol. 40, pp. 2003–2023, 2024, doi: 10.1109/TRO.2023.3347128.

[20] L. Brunke et al., "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, no. 1, pp. 411–444, 2022, doi: 10.1146/annurev-control-042920-020211.

[21] J. Yu et al., "A cooperative hunting method for multi-USVs based on trajectory prediction by OR-LSTM," *IEEE Trans. Veh. Technol.*, vol. 73, no. 12, pp. 18087–18101, 2024, doi: 10.1109/TVT.2024.3432739.

[22] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, PMLR 80, pp. 1861–1870, 2018. https://proceedings.mlr.press/v80/haarnoja18b.html

[23] J. Foerster et al., "Counterfactual multi-agent policy gradients," in *Proc. AAAI*, 2018. https://arxiv.org/abs/1705.08926

[24] V. Feinberg et al., "Model-based value estimation for efficient model-free reinforcement learning," 2018. https://arxiv.org/abs/1803.00101

[25] Open Source Robotics Foundation, "Virtual RobotX (VRX) simulation environment," software repository. https://github.com/osrf/vrx

[26] T. Johannink et al., "Residual reinforcement learning for robot control," arXiv:1812.03201, 2018. https://arxiv.org/abs/1812.03201